IAM-96 LAB: Python IAM Provisioning from Scratch — Raw Account Management API

> **Series:** IAM — IAM Administration | **Reference:** 96 — Python IAM Provisioning LAB | **Created:** July 2026 | **Last Updated:** 10/13/2026 | **Verified live:** 07/16/2026 — every sample response below is a real API response

Overview

Hands-on lab: build the complete IAM chain — **group → parameterized policy → boundary → binding** — from scratch in Python, using nothing but `requests` and the Account Management REST API. No SDK, no pre-built tool: every call is shown raw, with the **actual response** the API returned, so you can validate your own results against each step.

This is one of **three provisioning flavors**:

Flavor	Notebook	Best for
curl / shell	**IAM-12**	One-off scripts, CI glue
Terraform	**IAM-95**	Declarative IaC, drift detection, plan/apply review
Python raw API (this lab)	**IAM-96**	Learning the API itself; building your own tooling

What you will build (the BPN pattern from IAM-05 / IAM-10):

```
1. GROUP     PYLAB-Checkout Team
2. POLICY    PYLAB-tpl-team-data-access          (parameterized: ${bindParam:team})
3. BOUNDARY  PYLAB-bnd-team-checkout             (hardcoded scope, defense-in-depth)
4. BINDING   group → policy   at ENVIRONMENT level,
             with parameters={"team": "pylab-checkout"} + the boundary
```

Time: 30–40 minutes | **Difficulty:** Beginner-friendly | **Cost:** No consumption impact (IAM objects only)

Table of Contents

1. Step 0 — Setup
2. Step 1 — Get an OAuth token
3. Step 2 — Create the group
4. Step 3 — Create the parameterized policy
5. Step 4 — Create the boundary
6. Step 5 — Bind policy to group
7. Step 6 — Verify what you built
8. Step 7 — What happens on a re-run (idempotency)
9. Step 8 — Clean up
10. Troubleshooting
11. Validation checklist
12. References

Prerequisites

Requirement	Details
Python	3.10+ with `requests` (`pip install requests`)
Account UUID	Account Management → Account settings
OAuth client	Account Management → Identity & access management → OAuth clients
OAuth scopes	`account-idm-read account-idm-write iam-policies-management iam:policies:read iam:policies:write iam:bindings:read iam:bindings:write iam:boundaries:read iam:boundaries:write`

Requirement	Details
Prior reading	IAM-04 (policy authoring), IAM-05 (boundary design), IAM-10 (templated policies)

> ⚠️ **Safety rails (do not skip):**
> 1. Run only against a **sanctioned test tenant** — confirm the environment ID before executing.
> 2. Keep the `PYLAB-` name prefix so lab objects are unmistakable.
> 3. The binding is created at **environment level**. Groups and boundaries are account-level objects. Policies can be created at account **or** environment level (IAM-12 creates environment-level ones); this lab creates the policy at account level so one template can serve every environment. The binding is what grants access, and environment-scoping it confines the lab to one environment.

Step 0 — Setup

Export your credentials (or set them in your shell profile — never hardcode them in scripts):

```
export DT_ACCOUNT_UUID="&lt;your-account-uuid&gt;"
export DT_CLIENT_ID="dt0s02.XXXXXXXX"
export DT_CLIENT_SECRET="dt0s02.XXXXXXXX.YYYY..."
export DT_ENV_ID="abc12345"          # your TEST environment id
```

Start a Python session (`python3`) and set up the shared constants:

```
import json, os, requests

ACCOUNT = os.environ["DT_ACCOUNT_UUID"]
ENV_ID  = os.environ["DT_ENV_ID"]
API     = "https://api.dynatrace.com"
```

Step 1 — Get an OAuth token

The Account Management API does **not** use classic API tokens — it uses OAuth2 client credentials against the Dynatrace SSO, with your account as the `resource`:

```
resp = requests.post(
    "https://sso.dynatrace.com/sso/oauth2/token",
    data={
        "grant_type": "client_credentials",
        "client_id": os.environ["DT_CLIENT_ID"],
        "client_secret": os.environ["DT_CLIENT_SECRET"],
        "scope": "account-idm-read account-idm-write iam-policies-management "
                 "iam:policies:read iam:policies:write iam:bindings:read "
                 "iam:bindings:write iam:boundaries:read
iam:boundaries:write",
        "resource": f"urn:dtaccount:{ACCOUNT}",
    },
)
print(resp.status_code)
print(json.dumps(resp.json(), indent=2))

HEADERS = {"Authorization": f"Bearer {resp.json()['access_token']}",
           "Content-Type": "application/json"}
```

Sample result (HTTP 200):

```
{
  "scope": "account-idm-read account-idm-write iam-policies-management
iam:policies:read iam:policies:write iam:bindings:read iam:bindings:write
iam:boundaries:read iam:boundaries:write",
  "token_type": "Bearer",
  "expires_in": 300,
  "access_token": "eyJhbGciOiJFUzI1NiIs...(redacted)",
  "resource": "urn:dtaccount:&amp;lt;account-uuid&amp;gt;"
}
```

> **Note** `expires_in: 300` — tokens live 5 minutes. Long-running scripts must re-fetch.

✅ **Checkpoint:** HTTP 200 with an `access_token`. A 400 `invalid_scope` here means your OAuth client is missing one of the scopes.

Step 2 — Create the group

Groups live under `/iam/v1/accounts/{account}/groups`. **The payload is an ARRAY** — the most common first-timer error is posting a plain object:

```
resp = requests.post(
    f"{API}/iam/v1/accounts/{ACCOUNT}/groups",
    headers=HEADERS,
    json=[{
        "name": "PYLAB-Checkout Team",
        "description": "Lab: team-scoped access for checkout",
    }],
)
print(resp.status_code)
print(json.dumps(resp.json(), indent=2))

GROUP_UUID = resp.json()[0]["uuid"]
```

Sample result (HTTP 201):


```
[
  {
    "uuid": "0b7a0ab6-3e4a-4419-a70d-bdede67a0a56",
    "name": "PYLAB-Checkout Team",
    "owner": "LOCAL",
    "description": "Lab: team-scoped access for checkout",
    "hidden": false,
    "createdAt": null,
    "updatedAt": null
  }
]
```


✅ **Checkpoint:** response is an array with one object; save its `uuid`. `owner: "LOCAL"` means the group is managed here (SAML-federated groups show `"SAML"`).

Step 3 — Create the parameterized policy

One policy for ALL teams: the `${bindParam:team}` placeholders get resolved per binding (Step 5). Statement rules that matter (all live-verified — see IAM-04): **no wildcards**, comma-lists allowed, **reads** can be condition-scoped, storage **writes** cannot. `storage:buckets:read` is its own line because the policy reference says it is *"Required*

additionally to a table permission" — the table reads alone are not enough, and the security-context condition still decides which records match.

```
statement = """\
ALLOW environment:roles:viewer;
ALLOW app-engine:apps:run;
ALLOW document:documents:read;
ALLOW storage:logs:read, storage:spans:read, storage:metrics:read,
      storage:events:read, storage:bizevents:read
  WHERE storage:dt.security_context IN ("${bindParam:team}");
ALLOW storage:buckets:read;
ALLOW settings:objects:read WHERE settings:dt.security_context IN
("${bindParam:team}");
ALLOW settings:schemas:read;
"""

resp = requests.post(
    f"{API}/iam/v1/repo/account/{ACCOUNT}/policies",
    headers=HEADERS,
    json={
        "name": "PYLAB-tpl-team-data-access",
        "description": "Lab: parameterized team data access",
        "statementQuery": statement,
    },
)
print(resp.status_code)
print(json.dumps(resp.json(), indent=2))

POLICY_UUID = resp.json()["uuid"]
```

Sample result (HTTP 201, abridged) — note the API **parses your statement** into structured `statements[]`; this is where invalid permissions or wildcards get rejected with a 400:

```
{
  "uuid": "a20ff1ee-46fe-4a65-ac4a-fc922e5e4337",
  "name": "PYLAB-tpl-team-data-access",
  "description": "Lab: parameterized team data access",
  "tags": [],
  "statementQuery": "ALLOW environment:roles:viewer;\nALLOW app-
engine:apps:run;\n...",
  "statements": [
    { "effect": "ALLOW", "permissions": ["environment:roles:viewer"],
"conditions": null },
    { "effect": "ALLOW", "permissions": ["app-engine:apps:run"], "conditions":
null },
    { "effect": "ALLOW", "permissions": ["document:documents:read"],
"conditions": null },
    {
      "effect": "ALLOW",
      "permissions": [
        "storage:logs:read", "storage:spans:read", "storage:metrics:read",
        "storage:events:read", "storage:bizevents:read"
      ],
      "conditions": [
        {
          "name": "storage:dt.security_context",
          "operator": "IN",
          "values": ["${bindParam:team}"]
        }
      ]
    }
  ]
}
```


✅ **Checkpoint:** HTTP 201; the `statements[]` array shows your WHERE clause as a structured condition with the **literal** `${bindParam:team}` value — the placeholder survives until binding time.

Step 4 — Create the boundary

One standalone boundary per team, hardcoded scope (the policy carries the parameterized scope; the boundary is the defense-in-depth layer — IAM-05):

```
resp = requests.post(
    f"{API}/iam/v1/repo/account/{ACCOUNT}/boundaries",
    headers=HEADERS,
    json={
        "name": "PYLAB-bnd-team-checkout",
        "boundaryQuery": 'storage:dt.security_context IN ("pylab-
checkout");\n'
                         'settings:dt.security_context IN ("pylab-
checkout");\n',
    },
)
print(resp.status_code)
print(json.dumps(resp.json(), indent=2))

BOUNDARY_UUID = resp.json()["uuid"]
```

Sample result (HTTP 201):

```
{
  "uuid": "b410aff3-e405-48e4-b6c7-35634ecbb1dd",
  "levelType": "account",
  "levelId": "&lt;account-uuid&gt;",
  "name": "PYLAB-bnd-team-checkout",
  "boundaryQuery": "storage:dt.security_context IN (\"pylab-
checkout\");\nsettings:dt.security_context IN (\"pylab-checkout\");\n",
  "boundaryConditions": [
    { "name": "storage:dt.security_context",  "operator": "IN", "values":
["pylab-checkout"] },
    { "name": "settings:dt.security_context", "operator": "IN", "values":
["pylab-checkout"] }
  ],
  "metadata": {}
}
```

✅ **Checkpoint:** `levelType: "account"` — boundaries are always account-level objects; they take effect only when attached to a binding (next step). Add an `environment:management-zone IN ("...")` line only while the team still has a classic MZ (MZ2POL transitional).

Step 5 — Bind policy to group

The binding is where everything meets: it resolves `${bindParam:team}`, attaches the boundary, and — created at **environment** level — confines the granted access to one

environment:

```
resp = requests.post(
    f"{API}/iam/v1/repo/environment/{ENV_ID}/bindings/{POLICY_UUID}",
    headers=HEADERS,
    json={
        "groups": [GROUP_UUID],
        "parameters": {"team": "pylab-checkout"},
        "boundaries": [BOUNDARY_UUID],
    },
)
print(resp.status_code, repr(resp.text))
```

Sample result:

```
204 ''
```

> **Success is HTTP 204 with an EMPTY body** — don't call `.json()` on it. If you want account-wide grants instead, POST to `/iam/v1/repo/account/{ACCOUNT}/bindings/{POLICY_UUID}`.

✅ **Checkpoint:** 204. The user-visible effect: members of `PYLAB-Checkout Team` can now read only data whose `dt.security_context` is `pylab-checkout`, only in environment `abc12345`.

Step 6 — Verify what you built

Two useful reads. The quick one only lists policy UUIDs per group:

```
resp = requests.get(
    f"{API}/iam/v1/repo/environment/{ENV_ID}/bindings/groups/{GROUP_UUID}",
    headers=HEADERS,
)
print(json.dumps(resp.json(), indent=2))
```

Sample result (HTTP 200):

```
{
  "policyUuids": [
    "a20ff1ee-46fe-4a65-ac4a-fc922e5e4337"
  ]
}
```

The full listing carries the resolved **parameters and boundaries** — filter it for your group:

```
resp = requests.get(f"{API}/iam/v1/repo/environment/{ENV_ID}/bindings",
headers=HEADERS)
mine = [b for b in resp.json()["policyBindings"] if GROUP_UUID in
b.get("groups", [])]
print(json.dumps(mine, indent=2))
```

Sample result (HTTP 200):

```
[
  {
    "policyUuid": "a20ff1ee-46fe-4a65-ac4a-fc922e5e4337",
    "groups": ["0b7a0ab6-3e4a-4419-a70d-bdede67a0a56"],
    "parameters": {
      "team": "pylab-checkout"
    },
    "boundaries": ["b410aff3-e405-48e4-b6c7-35634ecbb1dd"]
  }
]
```

✅ **Checkpoint:** `parameters.team` equals your security context and exactly one boundary UUID is attached. Also check the UI: Account Management → Identity & access management → Policies → `PYLAB-tpl-team-data-access` → bindings tab shows the resolved value.

Step 7 — What happens on a re-run (idempotency)

POST the exact same binding from Step 5 again:

Sample result (HTTP 400):

```
{
  "error": {
    "code": 400,
    "message": "Policy binding cannot be created, such binding already exists:
level type: environment,level id: abc12345, policy uuid: a20ff1ee-46fe-4a65-
ac4a-fc922e5e4337, groups: [0b7a0ab6-3e4a-4419-a70d-bdede67a0a56], parameters:
{team=pylab-checkout}",
    "errorsMap": null
  }
}
```

> **This 400 is convergence, not failure.** When the desired state is identical to the current state, both the create endpoint and the per-group update endpoint (`POST .../bindings/{policy}/{group}`) answer "already exists." Any tooling you build on these calls should treat that message as success-unchanged — see IAM-12 §7.

Step 8 — Clean up

Deletion order matters: the group first (removing it drops its bindings), then the policy (a still-bound policy returns 400), then the boundary:

```
r1 = requests.delete(f"{API}/iam/v1/accounts/{ACCOUNT}/groups/{GROUP_UUID}",
headers=HEADERS)
r2 = requests.delete(f"
{API}/iam/v1/repo/account/{ACCOUNT}/policies/{POLICY_UUID}", headers=HEADERS)
r3 = requests.delete(f"
{API}/iam/v1/repo/account/{ACCOUNT}/boundaries/{BOUNDARY_UUID}",
headers=HEADERS)
print(r1.status_code, r2.status_code, r3.status_code)
```

Sample result:

```
200 204 204
```

> Quirk: group deletion answers `200` with body `1` (count of deleted groups); policy and boundary deletion answer `204` with empty bodies.

✅ **Checkpoint:** search `PYLAB-` in the UI — nothing remains.

Troubleshooting

HTTP	Message (abridged)	Cause	Fix
400	`invalid_scope` (token call)	OAuth client missing scopes	Recreate client with the full scope list
401	Unauthorized	Token expired (5-min lifetime)	Re-run Step 1
400	`payload.map is not a function`	Group POST got an object	Wrap payload in `[{...}]`
400	`ALLOW or DENY must be followed by a permission...`	Wildcard in statement	No wildcards — enumerate permissions
400	`Invalid condition name for permission storage:logs:write`	WHERE on a storage write	Storage writes take no conditions
400	`such binding already exists`	Re-posting identical binding	That's convergence — treat as success (Step 7)
400	on policy DELETE	Policy still bound	Delete the group first (Step 8 order)

Validation checklist

- [ ] Token response showed `expires_in: 300` and your account as `resource`
- [ ] Group create returned an **array** with `owner: "LOCAL"`
- [ ] Policy create echoed structured `statements[]` with the literal `${bindParam:team}` condition
- [ ] Boundary create showed `levelType: "account"` and parsed `boundaryConditions`
- [ ] Binding create returned **204 with an empty body**
- [ ] Full bindings list showed `parameters` + `boundaries` for your group
- [ ] Re-posting the binding returned 400 "already exists" (convergence)
- [ ] Cleanup returned `200, 204, 204` and the UI shows no `PYLAB-` objects

References

- **IAM-04** Policy Authoring (verified statement syntax) · **IAM-05** Boundary Design · **IAM-10** Templated Policy Assignments · **IAM-12** API Provisioning scripts (curl flavor) · **IAM-95** Terraform lab (same pattern as IaC)
- **MZ2POL series** — migrating off Management Zones onto this pattern
- Full-featured utility built on exactly these calls — dry-run CLI, idempotent re-runs, delete helpers: Python-IAM-Utility-3.0 (dtiam)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.